Components, projections, and delays

ar085 · 14 August 2026 · Draft

Contents

Developer guide

Components

Components are Python functions that add a reusable motif to a network. They disappear during compilation, leaving ordinary populations, projections, parameters, and groups.

`snn.components.ping` creates excitatory and inhibitory COBA-LIF populations with reciprocal E-to-I and I-to-E projections. Its defaults preserve established *tools/snn* numerical conventions. They are compatibility defaults, not a universal biological model.

```
cell = snn.components.ping(
    net,
    name="sensory",
    n_e=256,
    n_i=64,
    source=events,
)
```

A component may be instantiated several times. Larger circuits should be constructed by connecting named components rather than creating a new simulator class.

Projections

A projection declares its source, target port, synapse, weights, constraint, connection role, and optional delay.

```
net.connect(
    source.spikes,
    target.excitatory,
    name="source_to_target",
    synapse=snn.AMPA(tau=2 * snn.ms),
    weight=snn.Normal(0.2, 0.03),
    constraint=snn.NonNegative(),
    connection="feedforward",
    delay=0.2 * snn.ms,
)
```

The graph backend supports dense AMPA and GABA feedforward, recurrent, and feedback projections. Sparse matrices, structured connectivity, fractional-step delays, and modulatory synapses are not implemented.

Scheduling and causality
Feedforward edges with no delay follow a deterministic topological order. Recurrent and feedback spikes are causal: zero additional delay still means that a spike affects another population no earlier than the next simulation step. Positive delays must be exact multiples of the network timestep.

Zero-delay cycles, projection dimension errors, polarity mismatches, and invalid delays fail during planning rather than during simulation.

API reference

`components.ping`

```
snn.components.ping(
    net, *, name, n_e, n_i, source=None,
    tau_gaba=9 * snn.ms,
    include_silent_recurrence=False,
) -> PING
```

Returns a `PING` object with `.E` and `.I` populations. When `source` is supplied, the component adds a feedforward AMPA projection to E. The optional silent recurrent paths retain zero-valued E→E and I→I parameter shapes.

`Network.connect`

```
net.connect(
    source, target, *, name, synapse,
    weight=Constant(1.0), constraint=None,
    connection="feedforward", delay=None,
) -> Projection
```

`source` is a `Signal`; `target` is a population target-port string. Implemented connection values are `"feedforward"`, `"recurrent"`, and `"feedback"`. The authoring vocabulary also accepts `"modulatory"`, which the graph executor does not support. Synapse constructors are `AMPA(**values)`, `GABA(**values)`, `LeakyIntegrator(**values)`, and `Modulatory(**values)`.

`weight` accepts an initializer `Spec` or an existing `ParameterRef`. A new dense projection parameter has graph shape `[target, source]` and unit `nS`. `delay` is a time `Quantity` and must resolve to an integral number of graph timesteps for execution.

`Network.group`

```
with net.group(name, parent=None) as component:
    ...
```

The context records subsequently claimed members under one component. `parent` must name an existing group. Groups organize reports and diagrams; they do not change execution.

`Projection`

A `Projection` exposes `id`, `source`, `target`, `synapse`, `connection`, `delay`, `parameter_ids`, `group`, and `.weight`, which returns the first projection parameter as a `ParameterRef`.

Next: Compiling and executing bundles